Systems Programming and Computer Architecture

Laurin Seeholzer

October 22, 2026

Contents

1 C Programming **4**
- 1.1 History and Toolchain . . . 4
- 1.2 Control flow . . . 4
- 1.3 Basic types in C . . . 5
- 1.4 Operators . . . 6
- 1.5 Arrays . . . 6
- 1.6 Preprocessing . . . 7
- 1.7 Modularity . . . 8
- 1.8 Assertions . . . 9
- 1.9 Style . . . 9

2 Integers **10**
- 2.1 Operations . . . 10

3 Pointers **12**
- 3.1 Memory Layout and Virtual Address Space . . . 12
- 3.2 Pointer Fundamentals . . . 12
- 3.3 Pointer Operations . . . 13
- 3.4 Function Pointers . . . 13

4 Dynamic Memory Allocation **14**
- 4.1 Data Structures . . . 16
- 4.2 Memory Allocators . . . 17
- 4.3 Memory Pitfalls . . . 21

5 Design and Debugging **24**
- 5.1 Defects, Errors, and Failures . . . 24
- 5.2 Testing Principles . . . 24
- 5.3 Debugging Advice and Strategies . . . 25

6 Basic x86 Architecture and Assembly **28**
- 6.1 Computer Architecture and the x86-64 Model . . . 28
- 6.2 Assembly Syntax: AT&T vs. Intel . . . 29
- 6.3 Data Sizes and Register Organization . . . 29
- 6.4 Operand Specifiers and Memory Addressing Modes . . . 30
- 6.5 Data Movement Instructions . . . 31
- 6.6 Arithmetic and Logical Operations . . . 33
  - 6.6.1 Load Effective Address (`leaq`) . . . 33
  - 6.6.2 Unary and Binary Arithmetic Operations . . . 33
  - 6.6.3 Shift Operations . . . 34
- 6.7 Control Flow: Condition Codes, Jumps, and Loops . . . 35
  - 6.7.1 Condition Code Flags . . . 35
  - 6.7.2 Comparison and Testing . . . 35
  - 6.7.3 Reading Condition Codes (`set` Instructions) . . . 35
  - 6.7.4 Jumps and Branching . . . 36
  - 6.7.5 Conditional Moves (`cmov`) . . . 37
- 6.8 Procedures and the Calling Convention (System V ABI) . . . 37
  - 6.8.1 Argument Passing (The 6-Register Rule) . . . 37

6.8.2 Caller-Saved vs. Callee-Saved Registers 38
6.8.3 Procedure Control: `call` and `ret` 38
6.9 Stack Frames, Local Storage, and Buffer Overflows 39

7 Compiling C Control Flow and Procedures 41
7.1 Conditional Control Flow: Branching vs. Conditional Moves 41
7.1.1 Branching with Conditional Jumps 41
7.1.2 Branch Prediction and Pipeline Penalties 41
7.1.3 Conditional Moves (`cmov`) 42
7.2 Compiling Loops 43
7.2.1 Do-While Loops (Bottom-Tested) 43
7.2.2 While Loops: Jump-to-Middle vs. Guarded-Do 44
7.2.3 For Loops 45
7.3 Switch Statements and Multi-Way Branching 46
7.3.1 Jump Tables for Dense Cases 46
7.3.2 Sparse Switch Statements: Binary Decision Trees 48
7.4 Procedures and the System V AMD64 ABI 48
7.4.1 The Runtime Stack and Frame Architecture 48
7.4.2 Control Transfer: `call` and `ret` 49
7.4.3 Data Transfer: Arguments and Return Values 50
7.4.4 Caller-Saved vs. Callee-Saved Registers 50
7.4.5 Local Storage on the Stack and the Red Zone 51
7.4.6 Variadic Functions (`stdarg.h`) 52

1 C Programming

1.1 History and Toolchain

C is very fast, offers preprocessing and is close to metal. What you won't find in C are objects, classes, traits, features, methods, interfaces, fancy types, exception handling.

Important 1.1. Most importantly C does not offer memory managment, it uses pointers that offer direct access to memory adresses.

We'll use C99, the most commonly used variant, in this course.

Code Example 1.2

```
#include <studio.h>

int main(int argc, char *argv[])
{
    printf("hello, world\n");
    return 0;
}
```

1.2 Control flow

Code Example 1.3

```
if (Boolean expression) Statement_when_true
    else Statement_when_false

switch (Integer expression) {
    case Constant_1: Statement; break;
    ...
    default: Statement; break;
}

for (Initial; Test; Increment) Statement

while (Boolean expression) Statement

do Statement while (Boolean expression)

goto Label

printf()

return (Expression)
```

Important 1.4. Don't use goto: It's almost never a good idea, usually argued on performance.

Remark 1.5. goto can be used to break out of multiple nested loops or to cleanup nested code.

1.3 Basic types in C

Code Example 1.6

```
{static, const} type vairable = value;
```

Remark 1.7. The prefix **const** before a variable ensures it can not be changed lateron.

Definition 1.8. Scope: The scope of a variable in C is the Block it is definied in (lexically typed).

Remark 1.9. The prefix **static** before a variable definition ensures the value persists between function calls. Note that static limits the scope of a variable to the file/compilation unit.

Definition 1.10. Integers & Floats: Size of these differ from system to system

- **char** is always 1 Byte
- **int & float** are always 4 Bytes
- **long long & double** are always 8 Bytes

Definition 1.11. Booleans: Boolean values in C are integers where zero = false, non-zero = true

Definition 1.12. Void: Void is a type without a value and can be used for raw memory pointers or functions without returnvalue.

Definition 1.13. enums: used to enumerate/sequentially assign values to variables.

Code Example 1.14

```
eum { ThisIsZero, ThisIsOne, ThisIsTwo} numbers;
```

1.4 Operators

Code Example 1.15

```
() []
! ~ ++ -- + - * & (type) sizeof
* / %
+ -
<< >>
< <= > >=
== !=
&
^
|
&&
||
?:
= += -= *= /= %= &= ^= |= <<= >>=
,
```

Remark 1.16. Early termination in C means expressions such as A & B never evaluate B if A is already evaluated to false.

Definition 1.17. Ternery Operator: ternery operators allow for concise conditional assignment.

Code Example 1.18

```
int a = {test}? ifTrue : ifFalse;
```

Definition 1.19. Assignment Operator: In C an assignment to a variable is not just a statement its an expressison that evalueates to the value that is assigned.

Definition 1.20. Post- & Pre-Increment/Decement: As in many languages the difference between $a++$ and $++a$ is the value of the expression (before/after incrementing)

Important 1.21. Casting of most types to most other types in possible in C. What exactly happens is very complicated and determined by the types involved.

1.5 Arrays

Definition 1.22. Arrays are finite vectors of variables which all have the same type.

Important 1.23. The C-compiler does not check array bounds, always check them manualy!

Remark 1.24. Arrays do not have to be initialized when they are definied.

Code Example 1.25

```
int x[N] //uninitialized
int x[N] {} //initialized with zeros
```

Remark 1.26. Strings are initialized as an array of chars

Code Example 1.27

```
char s1[6] = "hello";
char s2[6] = {'h', 'e' , 'l', 'l', 'o', 0};
```

Remark 1.28. There are many libraries, to copy, compare, concatenate... strings in C. The language itself does not support any of these operations.

1.6 Preprocessing

Definition 1.29. Preprocessing replaces tokens in a C source file before compilation occurs. Text replacements and macros are managed using directives beginning with #.

Code Example 1.30

```
#define FOO BAZ       // Replaces FOO with BAZ
#define BAR(x) (x+3)  // Replaces BAR(x) with (x+3)
#undef FOO            // Undefines FOO; no replacements after this
    point
#define QUX           // Defines QUX as a null string
```

Remark 1.31. Complex multi-statement macros wrap code in a `do { ... } while(0)` loop to safely absorb trailing semicolons and prevent syntax errors when used in control flow statements (such as `if-else`).

Remark 1.32. Conditional compilation directives include or exclude blocks of code based on conditions evaluated by the preprocessor:

Code Example 1.33

```
#if expression
    ...
#elif expression
    ...
#else
    ...
#endif

#ifdef FOO     // Shorthand for #if defined(FOO)
#ifndef BAR    // Shorthand for #if !defined(BAR)
```

Remark 1.34. Macro parameters can be manipulated using special operators:

- `#c` (Stringifying): Converts a macro parameter into a string literal.
- `x ## y` (Token Pasting / Concatenation): Merges two tokens into a single token at compile-time.

Remark 1.35. The C preprocessor provides several standard predefined macros:

- `__FILE__`: Name of the current source file.
- `__LINE__`: Current line number.
- `__DATE__` / `__TIME__`: Date and time when the preprocessor runs.
- `__STDC__` / `__STDC_VERSION__`: Standard C compliance and version info.

1.7 Modularity

Definition 1.36. Declaration vs. Definition:

- A **declaration** specifies that an entity (function prototype, type, or global variable) exists somewhere without allocating memory for it.
- A **definition** provides the actual implementation or reserves memory for the variable/function.

Definition 1.37. A **module** consists of a public interface defined in a header file (`module.h`) containing declarations and macros, and an implementation in a source file (`module.c`) containing definitions and private declarations.

Remark 1.38. Declarations can be annotated with linkage modifiers:

- `extern`: The definition is located elsewhere (in another object module or compilation unit).
- `static`: Limits the visibility/scope of a function or global variable strictly to the current compilation unit.

Code Example 1.39

```
/* file.h */
#ifndef FILE_H
#define FILE_H

// Declarations, function prototypes, typedefs, and macros

#endif // FILE_H
```

1.8 Assertions

Definition 1.40. An **assertion** (`assert(expression)`) is a macro that evaluates a scalar condition at runtime. If the condition evaluates to false (`0`), it prints diagnostic details (file name, line number, expression) to standard error and terminates execution via `abort()`.

Remark 1.41. Assertions are tools for developers to detect program bugs and verify invariants during development. They should not be used to catch standard runtime or user errors (such as failed file opens). Assertions can be completely disabled in production builds by defining the `NDEBUG` macro (`-DNDEBUG`).

1.9 Style

Remark 1.42. Key guidelines for writing clean C code:

- Maintain clear formatting and consistent indentation to highlight block structure.
- Use explicit parentheses when in doubt to avoid operator precedence ambiguities.
- Use descriptive names for global entities and short names for local variables.
- Eliminate magic numbers using `const`, `enum`, or `#define` constants.
- Avoid function-like macros where regular inline functions suffice.

2 Integers

Lemma 2.1. Integers in C can be represented as **signed** (using two's complement) or **unsigned** (as a standard binary string).

Definition 2.2. Two's Complement: The most significant bit (MSB) has a negative weight (-2^{w-1}), while all other bits have positive weights. This allows representation of both positive and negative integers. To negate a number, invert all bits and add 1 ($\sim x + 1$).

Lemma 2.3. Ranges (w bits):

- Unsigned: 0 to $2^w - 1$
- Signed: -2^{w-1} to $2^{w-1} - 1$

Lemma 2.4. Casting between signed and unsigned does not change the underlying bit representation, only how the bits are interpreted.

Remark 2.5. In operations involving both a signed and an unsigned integer, the signed integer is automatically implicitly cast to unsigned.

2.1 Operations

Remark 2.6. Logic operations (`&`, `|`, `^`, `~`) operate bitwise. They behave as expected and return a signed integer if both operands are signed.

Definition 2.7. Shift Operations:

- **Left Shift** (`x << y`): Shifts bits left by y positions, filling with 0s on the right. Equivalent to multiplying by 2^y.
- **Logical Right Shift** (`x >> y`): Used for unsigned integers. Shifts right by y positions, filling with 0s on the left.
- **Arithmetic Right Shift** (`x >> y`): Used for signed integers. Shifts right by y positions, filling with the sign bit on the left.

Lemma 2.8. Addition: Signed and unsigned addition are performed at the bit level identically. If the result exceeds w bits, it overflows, and the extra bits are truncated.

Lemma 2.9. Multiplication: Exact multiplication requires $2w$ bits. In C, results are truncated to w bits. The bit-level operations for signed and unsigned multiplication yield the same truncated bit pattern.

Remark 2.10. Using Shift for Multiplication/Division:

- Multiplication by a power of 2 ($x \cdot 2^y$) can be optimized as `x << y`.
- Division by a power of 2 ($x/2^y$) can be optimized as `x >> y`. For signed negative integers, a bias is added before shifting to round towards zero: `(x + (1 << y) - 1) >> y`.

Lemma 2.11. Division: Integer division truncates towards zero. It is an expensive operation compared to addition or multiplication.

3 Pointers

3.1 Memory Layout and Virtual Address Space

Definition 3.1. The operating system provides each process with its own private virtual address space. Memory is organized into specific segments:

- **Kernel Virtual Memory:** Memory at the top of the address space that is inaccessible to user code.
- **User Stack:** Dynamically created region at runtime for managing function calls, growing downward.
- **Shared Libraries & Heap:** Memory-mapped region for shared libraries and dynamically allocated memory created via `malloc`.
- **Read/Write Segment:** Stores initialized and uninitialized global and static variables (`.data`, `.bss`) loaded from the executable.
- **Read-Only Segment:** Contains executable code and read-only constants (`.init`, `.text`, `.rodata`) loaded from the executable.

Remark 3.2. Set-up code allocates a stack frame upon entering a procedure to store local variables, return information, and temporary space. The space is deallocated by finish code upon return.

3.2 Pointer Fundamentals

Definition 3.3. A pointer is a variable whose value is the virtual memory address of another variable.

- **Address-of Operator (`&`):** Returns the memory address of a variable.
- **Dereference Operator (`*`):** Accesses or modifies the memory location referred to by a pointer.
- **Double Pointer:** A variable holding the memory address of another pointer (e.g., `int **dp`).
- **NULL Pointer:** A pointer assigned to address `0` (or `0x00000000`), representing a guaranteed invalid memory location.

Important 3.4. Any attempt to dereference a `NULL` pointer results in a segmentation fault.

Code Example 3.5

```
1 int x = 42;
2 int *p = &x;   // p holds the address of x
```

```
int **dp = &p; // dp holds the address of p
*p = 99;       // dereferences p to assign 99 to x
```

3.3 Pointer Operations

Definition 3.6. Pointer Arithmetic: Adding or subtracting an integer n to a pointer shifts the target memory address by $n \times$ sizeof(type) bytes.

Definition 3.7. Casting: Type casting a pointer reinterprets the byte-level data at that memory address according to the new type without changing the address itself.

Code Example 3.8

```
int arr[3] = {1, 2, 3};
int *int_ptr = &arr[0];
int_ptr += 1; // moves forward by sizeof(int) bytes to &arr[1]

char *char_ptr = (char *) int_ptr; // casts int* to char*
char_ptr += 1; // moves forward by sizeof(char) byte
```

3.4 Function Pointers

Definition 3.9. A function pointer stores the executable code address of a function, allowing functions to be passed as arguments or stored inside structures to implement polymorphism and object-oriented patterns.

Code Example 3.10

```
// Pointer to a function taking (int*, char) and returning int
int (*func)(int *, char);

// Example struct containing function pointers (vtable style)
struct file_operations {
ssize_t (*read) (struct file *, char *, size_t, loff_t *);
ssize_t (*write) (struct file *, const char *, size_t, loff_t *);
};
```

4 Dynamic Memory Allocation

Definition 4.1. Static vs Automatic Memory Allocation: Global variables are statically allocated when the program is loaded and deallocated when it exits, while function variables are automatically allocated on function call and deallocated on return.

Remark 4.2. Note that because automatically allocated memory is created on function calls, it is stored on the stack.

Remark 4.3. Often we need a larger amount of memory space that persists across multiple function calls without lasting the entire program lifetime.

Definition 4.4. Dynamic Memory Allocation: We can use allocation functions (such as `malloc`) to explicitly request dynamic memory blocks on the heap.

Code Example 4.5

```
1 char *a = (char *) malloc(10 * sizeof(char)); // Allocates
     uninitialized memory for 10 characters
2 char *zeros = (char *) calloc(10, sizeof(char)); // Allocates zero-
     initialized memory for 10 characters
3 free(a); // Frees allocated memory block
4
5 zeros = (char *) realloc(zeros, 20 * sizeof(char)); // Resizes
     allocated memory block to 20 characters
```

Definition 4.6. Memory Corruption: Occurs when a program unintentionally reads, modifies, or overwrites memory contents outside the bounds of an allocated block, or writes to memory that has already been deallocated or belongs to allocator metadata. This causes unpredictable program behavior, silent data corruption, or segmentation faults.

Definition 4.7. Memory Leaks: Occur when dynamically allocated heap memory is no longer referenced by any active pointer in the application but has not been returned to the heap via `free()`. Over time, uncollected memory accumulates and exhausts available memory.

Important 4.8. In C, memory management is strictly manual: every byte allocated on the heap must eventually be deallocated exactly once. Failing to properly manage memory boundaries and lifetimes leads to either memory corruption or memory leaks.

4.1 Data Structures

Definition 4.9. Structured Data is a custom type containing multiple data fields of different types. It is defined using the `struct` keyword.

Code Example 4.10

```
struct myData {
    int dataField;
    char dataName;
};
```

Definition 4.11. Unions overlay multiple member fields in a single shared memory location. Only one member value should be stored/interpreted at a time, and C performs no runtime tracking of the active member type.

Code Example 4.12

```
union currentTime {
    char stringName[10];
    int intTime;
    long longTime;
};
```

Remark 4.13. We can use `typedef` to define type aliases. Mixing struct tags and typedef aliases with the same name can cause confusion.

Remark 4.14. Tags (e.g., `struct tag {...}`) belong to a separate namespace from typenames; consequently, a typename and a tag can share the same identifier.

Remark 4.15. Structs and pointers allow building **dynamic data structures**. Using `void` pointers allows elements to be generic.

Code Example 4.16

```
struct listNode {
    void *value;
    struct listNode *next;
};

struct listNode *myList = NULL;
```

4.2 Memory Allocators

Definition 4.17. Implicit vs. Explicit Allocators:

- **Explicit Allocator:** The application program explicitly allocates and deallocates memory blocks (e.g., `malloc` and `free` in C, `new` and `delete` in C++).
- **Implicit Allocator (Garbage Collector):** The application explicitly requests memory allocation, but deallocation is handled automatically by the runtime system when a block is no longer in use or reachable (e.g., in Java, Python, Go, and Lisp).

Remark 4.18. Allocator Constraints and Requirements:

- **Arbitrary request sequences:** The allocator cannot make assumptions about the ordering or interleaving of allocation and deallocation requests.
- **Immediate response:** Requests must be serviced immediately; the allocator cannot buffer or reorder requests to optimize placement.
- **Heap storage only:** Any metadata structures used by the allocator must reside directly within the heap itself.
- **Alignment requirements:** Blocks must satisfy architecture-specific alignment boundaries (e.g., 8-byte alignment on 32-bit systems, 16-byte alignment on 64-bit systems).
- **Non-relocatable blocks:** Once allocated, blocks cannot be moved, compacted, or relocated by the allocator.

Definition 4.19. Performance Goals: Allocator efficiency is governed by two competing metrics:

- **Throughput:** The number of allocation and free operations completed per unit time (e.g., operations per second). High throughput requires low constant-time overhead for finding and freeing blocks.
- **Peak Memory Utilization (U_k):** Measures how efficiently the allocator utilizes heap memory. Given a sequence of requests $r_0, r_1, \ldots, r_k$, if P_k is the aggregate payload currently in use and H_k is the current heap size (high-water mark):
$$U_k = \frac{\max_{i \leq k} P_i}{H_k}$$
The objective is to maximize U_k (as close to 1 as possible).

Important 4.20. Throughput vs. Memory Utilization Trade-off: Maximizing memory utilization requires thorough searching to find tightly fitting free blocks and frequent coalescing (reducing throughput). Conversely, maximizing throughput

favors fast, greedy placement heuristics (like first-fit) that can cause fragmentation and degrade memory utilization.

Definition 4.21. Internal vs. External Fragmentation:

- **Internal Fragmentation:** Occurs when an allocated block is larger than the requested payload. Caused by allocator metadata overhead (headers, footers), alignment padding, or minimum block size constraints. It depends solely on individual request sizes and allocator overhead, making it easy to quantify.
- **External Fragmentation:** Occurs when there is sufficient aggregate free memory in the heap to satisfy an allocation request, but no single contiguous free block is large enough. It depends on the future history of requests and is significantly harder to measure and avoid.

Definition 4.22. Block Structure and Header: Because `free(ptr)` receives only a pointer without the block size, each block stores metadata preceding the payload:

- **Header:** A single word (4 or 8 bytes) placed immediately before the user payload containing:
  - **Block Size:** The total size of the block (including header, payload, and padding).
  - **Allocated Bit:** Since blocks are aligned to multiples of 8 (or 16) bytes, the lowest 3 (or 4) bits of any valid block size are always zero. The allocator repurposes the least significant bit (LSB) as an allocation flag: `1` for allocated, `0` for free.
- **Payload:** The user-accessible memory returned by `malloc`.
- **Padding:** Optional unused space to satisfy alignment constraints.

Code Example 4.23

```
// Header encoding example (8-byte alignment, 3 lowest bits free)
#define GET_SIZE(header)   ((header) & ~0x7) // Clears lowest 3 bits
#define GET_ALLOC(header)  ((header) & 0x1)  // Reads allocation status bit
#define PACK(size, alloc)  ((size) | (alloc)) // Combines size and status bit
```

Definition 4.24. Tracking Free Blocks: Dynamic allocators maintain free memory using different organizational strategies:

- **Implicit Free List:** All blocks (both allocated and free) are chained together implicitly through the size fields in their headers. Traversing blocks requires scanning sequentially from the beginning of the heap.

  - **Placement Policies:**
    - *First Fit:* Scans the list from the start and chooses the first free block that fits. Simple, but small fragments tend to accumulate at the start of the heap.
    - *Next Fit:* Starts searching where the previous search left off. Runs faster than first fit, but often leads to worse memory utilization.
    - *Best Fit:* Examines every free block and selects the one with the smallest fitting size. Maximizes utilization and minimizes fragmentation, but requires an exhaustive $O(N)$ scan.
  - **Coalescing & Boundary Tags:** When a block is freed, it must be merged with adjacent free blocks to prevent false fragmentation (consecutive small free blocks failing to satisfy a larger request). With **boundary tags** (a duplicate footer identical to the header placed at the end of each block), both previous and next adjacent blocks can be coalesced in $O(1)$ constant time.
- **Explicit Free List:** Links only the *free* blocks into an explicit doubly linked list by storing `prev` and `next` pointers inside the unused payload area of each free block.
  - Search time drops from $O(\text{all blocks})$ to $O(\text{free blocks})$.
  - **Insertion Policies:**
    - *LIFO (Last-In, First-Out):* Newly freed blocks are inserted at the head of the list in $O(1)$ time, but fragmentation can be worse.
    - *Address-Ordered:* Keeps the free list sorted by memory address. Freeing requires searching for the insertion point ($O(\text{free blocks})$), but it significantly reduces fragmentation and simplifies coalescing.
- **Segregated Free Lists:** Maintains an array of distinct free lists partitioned by size class (e.g., small exact classes like $16, 24, 32$ bytes, followed by power-of-two ranges like 33–64, 65–128, etc.).
  - Allocation searches only the list matching the requested size class, falling back to larger lists if necessary.
  - Combines the fast search time of simple explicit lists with the high memory utilization of best-fit approaches.

Definition 4.25. Garbage Collection (GC): An automated runtime technique that dynamically reclaims heap memory that will no longer be accessed by the application.

Definition 4.26. Memory as a Directed Graph: Garbage collection models heap memory as a directed reachability graph:

- **Nodes:** Memory blocks on the heap.
- **Edges:** Pointers from one block to another.
- **Roots:** Pointers stored outside the heap (CPU registers, local stack variables, global/static variables).
- **Reachable vs. Garbage:** A block is reachable if there is a directed path from any root. Blocks that cannot be reached from any root are garbage and can be safely reclaimed.

Definition 4.27. Classical Garbage Collection Algorithms:

- **Reference Counting:** Each block maintains a counter tracking incoming references. The block is freed immediately when the counter drops to zero. Cannot handle cyclic references without special cycle detection.
- **Mark-and-Sweep:** Tracing collector that identifies all reachable nodes by traversing from roots (mark phase), and then sweeps through all heap blocks to deallocate unmarked nodes (sweep phase).
- **Stop-and-Copy (Copying GC):** Splits heap memory into two regions (from-space and to-space). Reachable objects are copied contiguously into to-space, automatically compacting memory and eliminating external fragmentation.
- **Generational Collection:** Groups objects by lifetime into generations (young and old), collecting the short-lived young generation frequently and promoting surviving objects to the older generation.

Remark 4.28. Example: Mark-and-Sweep in Action:

- **Mark Phase:** Begins at the root set and traverses all pointers (e.g., via depth-first search), setting a mark bit in the header of each reached block.
- **Sweep Phase:** Performs a linear scan over all heap blocks. If a block's mark bit is clear, it is freed; if set, its mark bit is reset to 0 in preparation for the next cycle.

Code Example 4.29

```
// Pseudocode for Mark-and-Sweep Collection
void mark(ptr p) {
    if ((b = block_of(p)) != NULL && !is_marked(b)) {
        set_marked(b);
        for (each pointer child in payload(b))
            mark(child);
    }
}

void sweep() {
    ptr p = heap_start();
    while (p < heap_end()) {
```

```
13          if (!is_marked(p)) {
14              if (is_allocated(p))
15                  free(p);
16          } else {
17              clear_marked(p);
18          }
19          p = next_block(p);
20      }
21  }
```

Remark 4.30. Conservative GC for C: Because C is not type-safe, the runtime cannot distinguish whether an arbitrary memory word is a pointer or an integer. Conservative collectors (such as the Boehm-Demers-Weiser collector) assume that any word in the stack, registers, or heap whose numerical value corresponds to a valid heap address is a pointer, marking the referenced block as reachable to prevent premature freeing.

4.3 Memory Pitfalls

Remark 4.31. Because C allows direct pointer manipulation without automatic bounds checking or garbage collection, memory errors are common, difficult to diagnose, and can lead to severe security vulnerabilities.

Definition 4.32. Common Memory Pitfalls:

- **Dereferencing Bad Pointers:** Dereferencing an uninitialized pointer, a `NULL` pointer, or an arbitrary invalid address. This causes a segmentation fault (`SIGSEGV`) or corrupts arbitrary memory.
- **Reading Uninitialized Memory:** Assuming memory returned by `malloc()` is zero-filled. `malloc()` leaves the allocated block uninitialized with indeterminate garbage values. Memory must be initialized explicitly or allocated with `calloc()`.
- **Overwriting Memory (Buffer Overflow):** Writing past the boundaries of an allocated block due to off-by-one errors (e.g., `<=` instead of `<`), missing string null terminators, or calculating incorrect allocation sizes. This corrupts adjacent variables or heap metadata.
- **Referencing Nonexistent Variables:** Returning or storing the address of a local automatic stack variable. When the function returns, its stack frame is popped; the pointer becomes dangling, and subsequent function calls overwrite the data.
- **Freeing Blocks Multiple Times (Double Free):** Calling `free(p)` twice on the same pointer without an intervening allocation. This corrupts the allocator's free list data structures and can create exploitable security vulnerabilities.

- **Use-After-Free:** Accessing or modifying memory after it has been freed. The allocator may reallocate that block for another purpose, leading to silent data corruption or crashes.
- **Failing to Free Blocks (Memory Leaks):** Allocating memory on the heap and discarding all pointers to it without calling `free()`. Over time, leaked memory accumulates, exhausting system memory.

Code Example 4.33

```
// 1. Dereferencing Bad Pointers (e.g., passing val instead of &val)
int val;
scanf("%d", val); // Bug: passes garbage val as address (should be &val)

// 2. Reading Uninitialized Memory
int *arr = (int *) malloc(10 * sizeof(int));
int sum = arr[0]; // Bug: arr[0] contains indeterminate garbage values

// 3. Overwriting Memory (off-by-one loop boundary)
for (int i = 0; i <= 10; i++) // Bug: writes 11 elements (arr[10] is out of bounds)
    arr[i] = i;

// 4. Referencing Nonexistent Variables (dangling stack pointer)
int *get_bad_ref() {
    int local = 42;
    return &local; // Bug: local variable is deallocated when function returns!
}

// 5. Freeing Blocks Multiple Times (Double Free)
free(arr);
free(arr); // Bug: double free corrupts allocator free list metadata!

// 6. Referencing Freed Blocks (Use-After-Free)
int *ptr = (int *) malloc(sizeof(int));
free(ptr);
*ptr = 100; // Bug: writing to deallocated block

// 7. Failing to Free Blocks (Memory Leak)
void leak() {
    char *buf = (char *) malloc(1024);
    return; // Bug: pointer buf is lost; memory block is leaked
}
```

Remark 4.34. Best Practices for Safe Memory Management:

- Always set freed pointers to `NULL` (`free(p); p = NULL;`), as calling `free(NULL)` is guaranteed to be safe and harmless in C.

- Use automated memory debugging and dynamic analysis tools such as **Valgrind** or AddressSanitizer (`-fsanitize=address`) during development to detect leaks and invalid accesses at runtime.

5 Design and Debugging

5.1 Defects, Errors, and Failures

Definition 5.1. Defect, Error, and Failure Taxonomy:

- **Defect (Fault / Bug):** An imperfection, flaw, or mistake in the source code or design created by the programmer (e.g., an off-by-one loop condition, an uninitialized variable, or a missing boundary check).
- **Error:** An erroneous or invalid internal runtime state of the program resulting from the execution of a defect (e.g., an internal variable holding an illegal value or corrupted memory on the heap). An error is internal and may not yet be observable from outside the system.
- **Failure:** An externally observable divergence of the system's behavior from its specification or expected behavior (e.g., a segmentation fault / crash, incorrect output displayed to the user, an infinite loop, or data corruption).

Remark 5.2. The Fault Propagation Chain:

$$\text{Defect (Code)} \xrightarrow{\text{Execution}} \text{Error (State)} \xrightarrow{\text{Propagation}} \text{Failure (Symptom)}$$

- A defect remains dormant until the specific code path containing it is executed under conditions that trigger the flaw.
- An error remains hidden until it propagates to an observable boundary. An error may be masked, overwritten, or ignored without causing a failure.
- **The Debugging Inversion:** Debugging is the reverse process of failure propagation: start from an observed **failure**, trace backwards to identify the invalid internal **error** state, and isolate the underlying source code **defect**.

5.2 Testing Principles

Definition 5.3. Core Testing Strategies:

- **Test Early:** Testing should begin alongside development rather than being deferred until the end. Finding and resolving defects early reduces debugging complexity and cost.
- **Test Incrementally:** Build and test software in small, verifiable steps. When code is tested after every small addition, newly introduced bugs are localized to the most recent changes.
- **Test Small, Simple Units:** Isolate individual functions and components (unit testing) to verify their behavior independently before combining them into complex systems.
- **Test Boundary Conditions (Corner Cases):** Software bugs disproportion-

ately cluster around edges and extreme limits. Thorough test suites must evaluate:

  - *Zero / Empty Cases:* Empty strings (`""`), null pointers (`NULL`), collections of size 0.
  - *Singletons:* Arrays, buffers, or strings with exactly one element.
  - *Extremes and Limits:* Minimum and maximum integer values (`INT_MIN`, `INT_MAX`), buffer capacities ($N-1$, N, $N+1$).
  - *Off-by-One Boundaries:* Beginning and end of loops, arrays, and memory blocks.
  - *Invalid Inputs:* Negative values where positive are expected, out-of-range inputs, and malformed data.

- **Regression Testing:** Run automated test suites after every code change to ensure existing functionality remains unbroken.

Important 5.4. Dijkstra's Aphorism on Testing:

> *"Program testing can be used to show the presence of bugs, but never to show their absence!"* — Edsger W. Dijkstra (1970)

No matter how many test cases pass, testing cannot formally prove that a program is defect-free because testing all possible inputs and system states is computationally infeasible. Testing must be combined with disciplined design, modularity, assertions, and invariant verification.

5.3 Debugging Advice and Strategies

Definition 5.5. Systematic Debugging Guidelines:

- **Examine the Most Recent Change:** If a previously working system breaks, the bug almost certainly lies within the most recent modifications. Use version control tools (`git diff`, `git log -p`, `git bisect`) to isolate changes.
- **Simplify the Input (Minimal Reproducible Example):** Reduce large, complex failing inputs to the smallest, simplest possible test case that still triggers the failure. Eliminating irrelevant operations simplifies tracing program state.
- **Inspect the Stack Trace:** Upon a crash or assertion failure, use a debugger (e.g., `gdb` command `backtrace` or `bt`) or core dump to identify the exact call chain, active stack frames, and parameter values at the point of failure.
- **Sanity Check Intermediate Results:** Formulate concrete hypotheses about program state and verify them at key execution milestones using assertions (`assert()`), logging, or debugger watchpoints. Verify preconditions, postconditions, and loop invariants.

- **Understand the Root Cause:** Avoid superficial "band-aid" fixes (such as blindly changing `<` to `<=`, adding arbitrary `+1` offsets, or silencing warnings). Only modify code once the exact failure mechanism and the rationale for the fix are fully understood.

Code Example 5.6

```c
// Defensive programming: validating boundary conditions and invariants
#include <stdio.h>
#include <stdlib.h>
#include <assert.h>

int binary_search(const int *arr, int size, int target) {
    // 1. Sanity check preconditions & boundary conditions
    assert(arr != NULL || size == 0);
    assert(size >= 0);

    int low = 0;
    int high = size - 1; // Correct boundary: size - 1, not size

    while (low <= high) {
        // Prevent integer overflow: low + (high - low) / 2 instead of (low + high) / 2
        int mid = low + (high - low) / 2;

        // 2. Sanity check intermediate invariants
        assert(mid >= low && mid <= high);

        if (arr[mid] == target)
            return mid; // Target found
        else if (arr[mid] < target)
            low = mid + 1;
        else
            high = mid - 1;
    }

    return -1; // Target not found
}
```

Remark 5.7. Essential Systems Debugging Tools:

- **GDB (GNU Debugger):** Interactive debugger to step through execution (`step`, `next`), inspect variable values (`print`), set breakpoints/watchpoints, and inspect crash backtraces (`bt`).
- **Valgrind (Memcheck):** Dynamic memory analysis tool detecting memory leaks, invalid heap reads/writes, and use-after-free bugs.
- **Sanitizers (GCC / Clang):** Compile-time instrumentation adding automated checks:
  - AddressSanitizer (`-fsanitize=address`): Catches buffer overflows and use-

after-free bugs.

– UndefinedBehaviorSanitizer (`-fsanitize=undefined`): Catches undefined behavior (e.g., signed overflow, invalid shifts, null dereferences).

6 Basic x86 Architecture and Assembly

6.1 Computer Architecture and the x86-64 Model

Definition 6.1. Von Neumann Architecture: The foundational computing model consisting of three primary components interconnected by system buses:

- **Central Processing Unit (CPU):** Contains the **Control Unit** (fetches and decodes instructions), the **Arithmetic Logic Unit (ALU)** (executes arithmetic and logical operations), and the **Register File** (ultra-fast, low-latency internal storage).
- **Main Memory (RAM):** A linear sequence of byte-addressable storage holding both program instructions and runtime data.
- **Input/Output (I/O) System:** Interfaces the CPU and memory with external peripherals (disks, network, user interfaces).

Definition 6.2. The Programmer-Visible Machine State (x86-64): From the perspective of an assembly programmer or compiler, the state of an x86-64 processor comprises:

- **Program Counter (`%rip`):** The instruction pointer register storing the 64-bit virtual memory address of the next instruction to be fetched and executed.
- **General-Purpose Registers (GPRs):** 16 architectural 64-bit registers used to store integer data, function arguments, return values, and memory addresses.
- **Condition Code Flags (`RFLAGS`):** 1-bit status registers holding information about the arithmetic outcome of the most recently executed instruction (e.g., zero, negative, overflow, carry).
- **Vector / Floating-Point Registers:** Dedicated 128-bit to 512-bit registers (`%xmm0`–`%xmm15`, AVX `%ymm`, `%zmm`) for floating-point and SIMD operations.
- **Virtual Memory Space:** A byte-addressable array of memory locations accessed through 64-bit virtual addresses.

Remark 6.3. From C to Executable Machine Code: A high-level C program undergoes four distinct translation phases before execution:

$$\text{Source } (\texttt{.c}) \xrightarrow{\text{Preprocessor}} \text{Expanded } (\texttt{.i}) \xrightarrow{\text{Compiler } (\texttt{gcc -S})} \text{Assembly } (\texttt{.s})$$
$$\xrightarrow{\text{Assembler } (\texttt{as})} \text{Object } (\texttt{.o}) \xrightarrow{\text{Linker } (\texttt{ld})} \text{Executable Binary}$$

Assembly code is simply a human-readable textual representation of raw binary machine code instructions. While high-level languages abstract away hardware specifics (types, variables, expressions), assembly exposes the raw reality of hardware: fixed-size registers, memory addresses, condition codes, and explicit jump instructions.

6.2 Assembly Syntax: AT&T vs. Intel

Remark 6.4. AT&T Syntax vs. Intel Syntax: There are two major assembly syntax conventions for x86. The GNU toolchain on Linux (`gcc`, `as`, `gdb`, `objdump`) defaults to **AT&T syntax**, whereas Windows and Intel documentation utilize **Intel syntax**.

Feature	AT&T Syntax (GCC / Linux)	Intel Syntax
Operand Order	`instr source, destination`	`instr destination, source`
Register Prefix	Prefixed with `%` (e.g., `%rax`)	No prefix (e.g., `rax`)
Immediate Values	Prefixed with `$` (e.g., `$42`, `$0x10`)	Bare number (e.g., `42`, `10h`)
Memory Deref	`disp(base, index, scale)`	`[base + index*scale + disp]`
Instruction Suffix	Explicit size suffix (`b`, `w`, `l`, `q`)	Pointer prefix (`QWORD PTR`)
Example	`movq $8, (%rax, %rdx, 4)`	`mov QWORD PTR [rax+rdx*4], 8`

Important 6.5. In this course and throughout Linux systems programming, we strictly adhere to **AT&T syntax**:

$$\textbf{Mnemonic}\langle\text{size}\rangle \quad \textbf{Source}, \quad \textbf{Destination}$$

For example, in `movq %rax, %rbx`, the value flows from left to right: from `%rax` into `%rbx`.

6.3 Data Sizes and Register Organization

Definition 6.6. x86-64 Data Sizes: Originating from the 16-bit 8086 processor, Intel defines a **Word** as 16 bits (2 bytes). The size suffixes and corresponding C data types are:

- **Byte (`b`):** 8 bits (1 byte) $\longleftrightarrow$ C `char`
- **Word (`w`):** 16 bits (2 bytes) $\longleftrightarrow$ C `short`
- **Double Word (`l` for long):** 32 bits (4 bytes) $\longleftrightarrow$ C `int`, 32-bit integers, `float`
- **Quad Word (`q`):** 64 bits (8 bytes) $\longleftrightarrow$ C `long`, 64-bit pointers (`void*`), `double`

Definition 6.7. The 16 General-Purpose Registers: An x86-64 CPU provides 16 general-purpose 64-bit registers. For backward compatibility with 32-bit (IA32), 16-bit (8086), and 8-bit architectures, lower sub-portions of each register can be

accessed independently:

64-bit (q)	32-bit (l)	16-bit (w)	8-bit (b)	Primary Convention / Purpose
%rax	%eax	%ax	%al	Return value, Arithmetic accumulator
%rbx	%ebx	%bx	%bl	Callee-saved register
%rcx	%ecx	%cx	%cl	4th function argument, Shift count register
%rdx	%edx	%dx	%dl	3rd function argument, High 64-bit multiply/divide
%rsi	%esi	%si	%sil	2nd function argument, Source index in strings
%rdi	%edi	%di	%dil	1st function argument, Destination index
%rbp	%ebp	%bp	%bpl	Callee-saved, Optional frame pointer
%rsp	%esp	%sp	%spl	Stack pointer (points to current top of stack)
%r8	%r8d	%r8w	%r8b	5th function argument
%r9	%r9d	%r9w	%r9b	6th function argument
%r10	%r10d	%r10w	%r10b	Caller-saved scratch register
%r11	%r11d	%r11w	%r11b	Caller-saved scratch register
%r12	%r12d	%r12w	%r12b	Callee-saved register
%r13	%r13d	%r13w	%r13b	Callee-saved register
%r14	%r14d	%r14w	%r14b	Callee-saved register
%r15	%r15d	%r15w	%r15b	Callee-saved register

Important 6.8. The 32-bit Zero-Extension Rule: In x86-64, whenever an instruction generates a 32-bit register result (e.g., `movl`, `addl`, `xorl`), the CPU automatically **clears (sets to zero) the upper 32 bits** of the corresponding 64-bit register!

- `movl $0x12345678, %eax` sets the entire `%rax` to `0x0000000012345678`.
- Conversely, 8-bit and 16-bit operations (`movb`, `movw`) do **not** zero the upper bits; they preserve whatever was already present in the remaining higher bits of the register.

6.4 Operand Specifiers and Memory Addressing Modes

Definition 6.9. Operand Types: Most instructions take one or two operands classified into three types:

1. **Immediate ($Imm):** A constant numerical literal encoded directly into the instruction bytes (e.g., `$42`, `$-16`, `$0xFF`).
2. **Register (%reg):** The direct contents of one of the 16 registers (e.g., `%rax`,

`%edx`).

3. **Memory Access (...):** The value stored in memory at a computed virtual address, denoted by parentheses representing memory dereference.

Definition 6.10. General Addressing Form: The most general addressing mode in x86-64 computes the effective memory address using the formula:

$$\text{Address} = \text{Imm} + [r_b] + [r_i] \times s$$

written syntactically in AT&T format as:

$$\text{Imm}(r_b, r_i, s)$$

where:

- $\text{Imm} \in \mathbb{R}$: An optional constant offset (displacement).
- r_b: The base register (any 64-bit general-purpose register).
- r_i: The index register (any 64-bit register **except** `%rsp`).
- $s \in \{1, 2, 4, 8\}$: The scale factor (must be 1, 2, 4, or 8, exactly matching the byte size of primitive data types!).

Addressing Mode	Syntax	Effective Address	C Analogy
Absolute	`0x400050`	`0x400050`	Global variable
Indirect	`(%rax)`	`Mem[%rax]`	`*ptr`
Base + Displacement	`8(%rax)`	`Mem[%rax + 8]`	`ptr[1]` / field
Indexed	`(%rax, %rdx)`	`Mem[%rax + %rdx]`	`*(ptr + i)`
Scaled Indexed	`(%rax, %rdx, 4)`	`Mem[%rax + 4*%rdx]`	`int_array[i]`
Fully Scaled with Disp	`16(%rax, %rdx, 8)`	`Mem[%rax + 8*%rdx + 16]`	Array of structs

Code Example 6.11

```
// Examples of addressing in assembly
movq (%rdi), %rax          // Dereference pointer in %rdi: rax = *rdi
movl 8(%rdi), %eax         // Read 4-byte int at offset 8: eax = *(rdi + 8)
movq (%rdi, %rcx, 8), %rax // Array indexing: rax = rdi[rcx] (array of 8-byte longs)
movl -16(%rbp), %eax       // Access local variable on the stack frame
```

6.5 Data Movement Instructions

Definition 6.12. The `mov` Family: Copies data from a source operand to a destination operand without altering the source:

`mov`⟨size⟩ Source, Destination

Available variants include `movb` (1 byte), `movw` (2 bytes), `movl` (4 bytes), and `movq` (8 bytes).

Important 6.13. The Memory-to-Memory Prohibition: An x86-64 instruction **cannot** perform both a memory read and a memory write simultaneously. Therefore, an instruction of the form:

`movq (%rax), (%rdx)` **IS ILLEGAL IN x86-64**

To copy data from one memory location to another, the value must first be loaded into an intermediate register: **Code Example 6.14**

```
1 movq (%rax), %rcx    // Load from memory into temporary
     register %rcx
2 movq %rcx, (%rdx)    // Store from %rcx to destination memory
```

Definition 6.15. Extension Instructions (Moving Smaller to Larger Destinations): When moving data from a smaller source to a larger destination, the missing upper bits must be populated:

- **Zero Extension (`movz`):** Fills remaining high-order bits with zeros (used for **unsigned** data types):
  - `movzbl` (byte to doubleword), `movzbq` (byte to quadword), `movzwq` (word to quadword).
  - Note: `movzlq` does not exist because standard `movl` already zeroes the upper 32 bits!
- **Sign Extension (`movs`):** Replicates the most significant bit (the sign bit) into all upper bits (used for **signed** two's-complement data types):
  - `movsbl`, `movsbq`, `movswq`, `movslq`.
  - Special instruction: `cltq` sign-extends `%eax` into the entire 64-bit `%rax`.

Definition 6.16. Stack Operations (`pushq` and `popq`): The runtime stack is managed by the stack pointer register `%rsp`, which points to the current top of the stack. Because the x86 stack **grows downward** towards lower numerical memory addresses:

- **`pushq Src`:** Pushes an 8-byte value onto the stack:

$$\texttt{\%rsp} \leftarrow \texttt{\%rsp} - 8, \quad \text{Mem}[\texttt{\%rsp}] \leftarrow \text{Src}$$

- **`popq Dst`:** Pops the top 8-byte value from the stack into a destination register:

$$\text{Dst} \leftarrow \text{Mem}[\texttt{\%rsp}], \quad \texttt{\%rsp} \leftarrow \texttt{\%rsp} + 8$$

6.6 Arithmetic and Logical Operations

6.6.1 Load Effective Address (`leaq`)

Definition 6.17. Load Effective Address (`leaq`): The `leaq` instruction shares the exact memory-addressing syntax as `movq`, but with one crucial difference: **it does not access memory!**

$$\texttt{leaq} \quad \text{Imm}(r_b, r_i, s), \quad \text{Dst}$$

Instead of reading from memory, `leaq` computes the address calculation $\text{Imm} + [r_b] + [r_i] \times s$ and stores the resulting numerical value directly into `Dst`.

Remark 6.18. The Dual Nature and Compiler Optimization of `leaq`: Compilers use `leaq` for two distinct purposes:

1. **Generating Pointers:** To compute memory addresses (equivalent to the C address-of operator `&`), such as obtaining the address of a struct field: `leaq 8(%rdi), %rax`.
2. **Fast Integer Arithmetic:** Because modern CPUs have dedicated hardware address generation units (AGUs) that compute base + index × scale in a single clock cycle, compilers hijack `leaq` to perform integer math without memory involvement.

Code Example 6.19

```
// Fast arithmetic tricks performed by the compiler via leaq:
leaq (%rdi, %rdi, 2), %rax   // rax = rdi + 2*rdi = 3 * x
leaq (%rdi, %rdi, 4), %rax   // rax = rdi + 4*rdi = 5 * x
leaq 7(%rdi, %rsi),   %rax   // rax = rdi + rsi + 7 = x + y + 7
leaq (%rdi, %rdi, 8), %rax   // rax = rdi + 8*rdi = 9 * x
```

Important 6.20. Unlike standard arithmetic instructions (`addq`, `subq`), the `leaq` instruction **never alters condition code flags**!

6.6.2 Unary and Binary Arithmetic Operations

Instruction	Syntax	Operation Performed	Effect on Flags
`incq D`	`incq %rax`	$D \leftarrow D + 1$	Sets ZF, SF, OF (preserves CF!)
`decq D`	`decq %rax`	$D \leftarrow D - 1$	Sets ZF, SF, OF (preserves CF!)
`negq D`	`negq %rax`	$D \leftarrow -D$ (two's complement)	Sets all flags
`notq D`	`notq %rax`	$D \leftarrow\sim D$ (bitwise NOT)	Does not affect flags
`addq S, D`	`addq %rcx, %rax`	$D \leftarrow D + S$	Sets all flags (CF, ZF, SF, OF)
`subq S, D`	`subq %rcx, %rax`	$D \leftarrow D - S$	Sets all flags (computes $D - S$)
`imulq S, D`	`imulq %rcx, %rax`	$D \leftarrow D \times S$	Sets CF, OF on overflow
`xorq S, D`	`xorq %rax, %rax`	$D \leftarrow D \oplus S$	Clears CF, OF; sets ZF, SF
`andq S, D`	`andq %rcx, %rax`	$D \leftarrow D \,\&\, S$	Clears CF, OF; sets ZF, SF
`orq S, D`	`orq %rcx, %rax`	$D \leftarrow D \mid S$	Clears CF, OF; sets ZF, SF

Remark 6.21. Subtleties of `subq` and `xorq`:

- **Operand Order in `subq`:** Remember AT&T order: `subq %rcx, %rax` computes:
$$\texttt{\%rax} \leftarrow \texttt{\%rax} - \texttt{\%rcx}$$
The second operand is the minuend (the value being subtracted from).
- **Zeroing Registers:** The standard, idiomatically optimal way to zero a register is `xorl %eax, %eax`. This requires fewer instruction bytes than `movq $0, %rax` and is recognized by CPU decoders as a zero-latency register renaming operation.

6.6.3 Shift Operations

Definition 6.22. Bit Shift Instructions: Bit shifts shift the bits of destination D by a shift count k:

- **Left Shift (`salq` / `shlq`):** Shifts left, zero-filling the vacated lower bits. (Equivalent to multiplying by 2^k).
- **Arithmetic Right Shift (`sarq`):** Shifts right, replicating the sign bit into vacated high-order bits. (Preserves two's complement sign for signed integer division by 2^k).
- **Logical Right Shift (`shrq`):** Shifts right, zero-filling the vacated high-order bits. (Used for unsigned division by 2^k).

Important 6.23. The Shift Count Constraint: The shift count operand can be an immediate constant (e.g., `salq $3, %rax`). However, if the shift count is stored in a register, x86-64 **strictly mandates** that it must reside in the single register `%cl` (the low 8-bit byte of `%rcx`)! No other register can be used as a variable shift counter. For 64-bit quadwords, only the lower 6 bits of `%cl` are read ($0 \leq k \leq 63$).

6.7 Control Flow: Condition Codes, Jumps, and Loops

6.7.1 Condition Code Flags

Definition 6.24. Hardware Condition Codes (Status Flags): The CPU maintains a set of single-bit status flags updated automatically by most arithmetic instructions:

- **CF (Carry Flag):** Set if the most recent operation generated a carry out of the most significant bit. Used to detect **unsigned overflow**.
- **ZF (Zero Flag):** Set if the most recent operation yielded a result of **zero** (i.e., Result == 0).
- **SF (Sign Flag):** Set if the most recent operation yielded a **negative** result (the most significant bit is 1).
- **OF (Overflow Flag):** Set if the most recent operation caused a **signed two's-complement overflow** (e.g., adding two positive numbers yields a negative result, or subtracting a negative from positive overflows).

6.7.2 Comparison and Testing

Definition 6.25. Comparison (`cmp`) and Test (`test`): These instructions set condition flags without modifying any data registers:

- **`cmpq S2, S1`:** Computes the difference $S_1 - S_2$ purely to set flags. It acts identically to `subq S2, S1`, except the difference is discarded:
  - If $S_1 == S_2$: ZF is set.
  - If $S_1 < S_2$ (signed): SF $\neq$ OF.
  - If $S_1 < S_2$ (unsigned): CF is set.
- **`testq S2, S1`:** Computes the bitwise conjunction S_1 & S_2 purely to set flags. It acts identically to `andq S2, S1`, except the result is discarded:
  - Most commonly used with identical operands: `testq %rax, %rax` sets ZF if `%rax` is zero, and SF if `%rax` is negative.

6.7.3 Reading Condition Codes (`set` Instructions)

Definition 6.26. The `set` Family: Sets a single byte destination register (e.g., `%al`) to 1 if the specified condition holds, and 0 otherwise:

- `sete` / `setz` ($ZF = 1$): Set on equal / zero ($S_1 == S_2$).
- `setne` / `setnz` ($ZF = 0$): Set on not equal / non-zero ($S_1 \neq S_2$).
- **Signed Comparisons:**
  - `setl` (less: $SF \oplus OF = 1$), `setle` (less or equal: $(SF \oplus OF) \mid ZF = 1$).
  - `setg` (greater: $\sim (SF \oplus OF)$ & $\sim ZF = 1$), `setge` (greater or equal: $\sim (SF \oplus OF) = 1$).
- **Unsigned Comparisons:**
  - `setb` (below / carry: $CF = 1$), `setbe` (below or equal: $CF \mid ZF = 1$).
  - `seta` (above: $\sim CF$ & $\sim ZF = 1$), `setae` (above or equal: $\sim CF = 1$).

6.7.4 Jumps and Branching

Definition 6.27. Jump Instructions (`jmp` and `j*`): Alter execution by loading a new address into the program counter `%rip`:

- **Unconditional Jump (`jmp Target`):** Jumps directly to `Target`. Can be direct (`jmp .L2`) or indirect via register/memory (`jmp *%rax`).
- **Conditional Jumps (`j* Target`):** Jump only if specific condition flags are set; otherwise, execution falls through to the next sequential instruction.

Jump Instruction	Condition Checked	Description
`jmp`	None	Unconditional branch
`je` / `jz`	$ZF = 1$	Equal / Zero ($a == b$)
`jne` / `jnz`	$ZF = 0$	Not Equal / Not Zero ($a \neq b$)
`js` / `jns`	$SF = 1$ / $SF = 0$	Negative / Non-negative
`jg` / `jge`	Signed: $>$ / $\geq$	Greater / Greater or Equal
`jl` / `jle`	Signed: $<$ / $\leq$	Less / Less or Equal
`ja` / `jae`	Unsigned: $>$ / $\geq$	Above / Above or Equal
`jb` / `jbe`	Unsigned: $<$ / $\leq$	Below / Below or Equal

Code Example 6.28

```
// C code:
long max(long a, long b) {
    if (a > b) return a;
    else return b;
}

// Corresponding x86-64 assembly:
max:
    cmpq %rsi, %rdi      // Compare a (%rdi) and b (%rsi) by
        computing rdi - rsi
    jle  .L_b_is_larger  // If a <= b, jump to .L_b_is_larger
    movq %rdi, %rax      // Otherwise, return a (%rdi)
    ret
```

```
13 .L_b_is_larger:
14     movq %rsi, %rax      // Return b (%rsi)
15     ret
```

6.7.5 Conditional Moves (`cmov`)

Definition 6.29. Conditional Move (`cmov`): Copies data from source to destination only if condition flags satisfy the specified predicate:

$$\texttt{cmov}\langle\text{condition}\rangle \quad \text{Source}, \quad \text{Destination}$$

If the condition is false, the instruction acts as a no-op (`nop`).

Remark 6.30. Intuition & Branch Prediction Optimization: Modern high-performance processors rely on deep pipelining (15–20 pipeline stages). To keep the pipeline full, the processor employs a **branch predictor** to speculate which way a conditional jump will go before the condition is evaluated.

- When a branch is predicted correctly, execution proceeds without delay.
- When a branch is **mispredicted**, the entire instruction pipeline must be flushed, throwing away partially executed instructions and costing **15–30 wasted clock cycles**.

A **conditional move (`cmov`)** computes both potential outcomes and then conditionally selects the result using data flow rather than control flow jumps. Because there are no jump instructions, there is **zero branch misprediction penalty**!

Code Example 6.31

```
1 // Optimizing max() with conditional move:
2 max:
3     movq %rsi, %rax      // rax = b
4     cmpq %rsi, %rdi      // Compare a (%rdi) and b (%rsi)
5     cmovg %rdi, %rax     // If a > b, overwrite rax with a; else
          keep b
6     ret                  // Done! No branching, no pipeline stall!
```

6.8 Procedures and the Calling Convention (System V ABI)

Definition 6.32. The System V AMD64 Calling Convention: The standard Application Binary Interface (ABI) used on Linux and macOS defining how compiled functions interact, pass parameters, return values, and manage registers.

6.8.1 Argument Passing (The 6-Register Rule)

Theorem 6.33. Passing Function Arguments in Registers: Under the System V ABI, the first 6 integer or pointer arguments to a function are passed directly inside registers in strict order:

1. 1st Argument: `%rdi`
2. 2nd Argument: `%rsi`
3. 3rd Argument: `%rdx`
4. 4th Argument: `%rcx`
5. 5th Argument: `%r8`
6. 6th Argument: `%r9`

Any additional arguments (7th, 8th, etc.) are pushed onto the runtime stack in reverse order. The primary return value is returned in `%rax` (and `%rdx` for 128-bit results).

Remark 6.34. Mnemonic to Remember Arguments: A famous mnemonic to memorize the argument register sequence is:

rDI*ane's* **rSI***lk* **rDX***ress* **rCX***ost* **r8**9 (`%rdi`, `%rsi`, `%rdx`, `%rcx`, `%r8`, `%r9`)

6.8.2 Caller-Saved vs. Callee-Saved Registers

Definition 6.35. Register Preservation Roles: To ensure functions do not corrupt each other's local state, registers are strictly divided into two categories:

- **Caller-Saved Registers (Call-Clobbered):**

  `%rax`, `%rdi`, `%rsi`, `%rdx`, `%rcx`, `%r8`, `%r9`, `%r10`, `%r11`

  These registers can be freely modified by the called function (*callee*). If the calling function (*caller*) needs their values preserved across a `call`, the caller must save them on the stack prior to making the call.

- **Callee-Saved Registers (Preserved Registers):**

  `%rbx`, `%rbp`, `%r12`, `%r13`, `%r14`, `%r15`, `%rsp`

  The called function (*callee*) is strictly obligated to preserve these registers. If the callee intends to use any of them, it **must save their original values** (typically via `pushq`) upon entry, and **restore them** (via `popq`) before returning with `ret`.

6.8.3 Procedure Control: `call` and `ret`

Definition 6.36. Mechanics of `call` and `ret`: Functions transfer control through the stack:

- **`call Target`:**
  1. Pushes the 8-byte **return address** (the memory address of the instruction immediately following the `call`) onto the stack:
  $$\texttt{\%rsp} \leftarrow \texttt{\%rsp} - 8, \quad \text{Mem}[\texttt{\%rsp}] \leftarrow \texttt{\%rip}_{\text{next}}$$
  2. Jumps to the target function: `%rip` $\leftarrow$ Target.
- **`ret`:**
  1. Pops the return address off the top of the stack:
  $$\texttt{\%rip} \leftarrow \text{Mem}[\texttt{\%rsp}], \quad \texttt{\%rsp} \leftarrow \texttt{\%rsp} + 8$$
  2. Resumes execution at the retrieved address.

Important 6.37. The 16-Byte Stack Alignment Requirement: The System V AMD64 ABI requires that the stack pointer `%rsp` must be a multiple of 16 (16-byte aligned) immediately before any `call` instruction is issued. When a `call` executes, it pushes the 8-byte return address, leaving `%rsp` offset by 8 upon function entry. Functions must ensure their stack allocations preserve this 16-byte alignment before calling any sub-functions!

6.9 Stack Frames, Local Storage, and Buffer Overflows

Definition 6.38. Stack Frame Anatomy: Each function invocation allocates a contiguous region of memory on the stack called a **stack frame**:

- **Top of Frame (High Address):** 7th+ incoming arguments passed by caller, followed by the 8-byte Return Address.
- **Saved Frame Pointer (`%rbp`):** (Optional) Preserved base pointer of the caller.
- **Saved Callee-Saved Registers:** Registers preserved by the function.
- **Local Variables:** Local arrays or structures that cannot fit entirely inside registers.
- **Outgoing Arguments (Low Address / `%rsp`):** Space for arguments passed to called subroutines.

Code Example 6.39

```
// Standard Function Prologue and Epilogue:
function_example:
    pushq %rbp              // Save caller's base pointer
    movq %rsp, %rbp         // Set up new base pointer for current
        frame
    subq $32, %rsp          // Allocate 32 bytes for local stack
        variables
```

```
6
7       // ... Function Body ...
8
9       movq %rbp, %rsp         // Deallocate local storage (or use '
            leave')
10      popq %rbp               // Restore caller's base pointer
11      ret                     // Pop return address and return to
            caller
```

Definition 6.40. Buffer Overflow (Stack Smashing): Occurs when a program writes data beyond the allocated boundary of a stack buffer (e.g., using unsafe functions like `gets()` or `strcpy()`). Because the runtime stack grows downward, local buffers are located below the saved return address. Overflowing the buffer writes upwards, overwriting the saved frame pointer and the **saved return address**. When `ret` executes, the CPU pops the attacker-controlled address into `%rip`, hijacking program control flow.

Definition 6.41. Modern Stack Protection Defenses:

- **Stack Canaries (`-fstack-protector`):** The compiler inserts a random secret value (canary) from `%fs:40` between local stack buffers and the saved return address. Before executing `ret`, the function compares the canary against `%fs:40`. If a buffer overflow altered the canary, the program aborts with a stack smashing crash rather than executing attacker code.
- **No-Execute (NX / W^X Bit):** Memory pages containing the stack are marked as read-write but strictly non-executable, preventing injected shellcode on the stack from executing.
- **Address Space Layout Randomization (ASLR):** Randomizes the base addresses of the stack, heap, and shared libraries upon every execution, making it difficult for attackers to predict fixed code injection targets.

7 Compiling C Control Flow and Procedures

7.1 Conditional Control Flow: Branching vs. Conditional Moves

Definition 7.1. Control Flow Representation: In high-level C, execution paths are structured using conditionals (`if-else`, the ternary operator `? :`, and `switch`) and loops (`while`, `do-while`, and `for`). In x86-64 machine assembly, the processor executes instructions sequentially as dictated by the Program Counter (`%rip`). Control flow is altered through two fundamental mechanisms:

- **Control Transfers (Jumps / Branching):** Modifying `%rip` conditionally or unconditionally based on status flags in `RFLAGS` (e.g., `jmp`, `je`, `jl`, `jg`).
- **Conditional Data Transfers (Conditional Moves):** Conditionally transferring data between registers (`cmov*`) without altering the sequential instruction stream.

7.1.1 Branching with Conditional Jumps

Definition 7.2. Jump-Based Conditional Execution: The standard translation of a C `if (test) then-statement else else-statement` evaluates the condition, sets condition codes using `cmp` or `test`, and executes a conditional jump based on the outcome:

Code Example 7.3

```
// C code:
long absdiff(long x, long y) {
    if (x < y) {
        return y - x;
    } else {
        return x - y;
    }
}

// x86-64 Assembly (Branch-based):
// Arguments: x in %rdi, y in %rsi
absdiff:
    cmpq    %rsi, %rdi          // Compare x and y (computes x - y)
    jl      .L_then             // If x < y, branch to .L_then
    movq    %rdi, %rax          // Else-path: %rax = x
    subq    %rsi, %rax          // %rax = x - y
    ret                         // Return result in %rax
.L_then:
    movq    %rsi, %rax          // Then-path: %rax = y
    subq    %rdi, %rax          // %rax = y - x
    ret                         // Return result in %rax
```

7.1.2 Branch Prediction and Pipeline Penalties

Definition 7.4. Instruction Pipelining and Branch Prediction: Modern high-performance x86 processors employ deep execution pipelines (typically 15 to 20 stages)

to execute multiple instructions per cycle.

- **Branch Predictor:** When fetching a conditional jump instruction (`j*`), the processor cannot afford to wait until the condition code is resolved in the ALU stages. It employs a hardware **Branch Predictor** with history tables to speculate whether the branch will be taken or not taken.
- **Speculative Execution:** The CPU speculatively fetches and decodes instructions along the predicted path.
- **Branch Misprediction Penalty:** If the branch prediction turns out to be wrong once flags are evaluated, all speculatively fetched instructions in the pipeline must be discarded and flushed. This incurs a severe performance penalty of **15 to 30 clock cycles** of lost work.

7.1.3 Conditional Moves (`cmov`)

Definition 7.5. Conditional Move Translation: Rather than changing the execution path, a compiler can compute both candidate outcomes eagerly and use a **conditional move** (`cmov`) instruction to copy the chosen result into the destination register:

$$\texttt{cmov}\langle\text{condition}\rangle \quad \text{Source}, \quad \text{Destination}$$

The transfer takes place if and only if the specified condition code holds; otherwise, the destination register remains untouched.

Code Example 7.6

```
// x86-64 Assembly (Conditional Move):
// Arguments: x in %rdi, y in %rsi
absdiff:
    movq    %rsi, %rdx
    subq    %rdi, %rdx          // %rdx = y - x (then-expression)
    movq    %rdi, %rax
    subq    %rsi, %rax          // %rax = x - y (else-expression,
        default in %rax)
    cmpq    %rsi, %rdi          // Compare x and y
    cmovl   %rdx, %rax          // If x < y, %rax = %rdx (y - x);
        else keep x - y
    ret
```

Remark 7.7. Branch vs. Conditional Move Trade-offs:

- **Data Dependency vs. Control Dependency:** Conditional moves transform a *control dependency* (which instruction to execute next) into a *data dependency* (which operand to select). The instruction flow is strictly linear and deterministic.
- **Unpredictable Branches:** When branches are random (e.g., branching on unsorted array elements), branch predictors fail roughly 50% of the time. Here, `cmov` yields substantial speedups by eliminating all pipeline flushes.

- **Predictable Branches:** If a branch is highly predictable (e.g., error checking that rarely triggers or loops with hundreds of iterations), branch-based code is faster because it computes only the taken branch.

Important 7.8. When Conditional Moves CANNOT or MUST NOT be Used: Compilers are prohibited from emitting `cmov` in the following critical scenarios:

1. **Memory Safety / Null-Pointer Dereferencing:**

```
long val = (p != NULL) ? *p : 0;
```

   Because `cmov` requires evaluating both paths before selection, computing `*p` eagerly dereferences `NULL` when `p == NULL`, triggering a fatal Segmentation Fault (`SIGSEGV`).

2. **Side Effects:**

```
long val = (x > 0) ? x++ : y--;
```

   Eagerly evaluating both sides executes both `x++` and `y--`, corrupting program state regardless of the condition.

3. **Expensive Computations:** If evaluating both candidate expressions requires expensive calculations or function calls (e.g., `f(x) ? heavy1() : heavy2()`), computing both wastes more CPU cycles than an occasional branch misprediction.

4. **Arithmetic Exceptions:**

```
long val = (y != 0) ? (x / y) : 0;
```

   Eagerly evaluating the division when $y = 0$ triggers a hardware divide-by-zero trap (`SIGFPE`).

7.2 Compiling Loops

Definition 7.9. Loop Compilation Principle: In assembly language, loops are synthesized by combining conditional and unconditional jumps with condition code evaluations. Repetitive execution occurs when a jump targets an earlier address in the instruction stream.

7.2.1 Do-While Loops (Bottom-Tested)

Definition 7.10. Do-While Translation: A `do-while` loop inherently matches the hardware execution model:

1. The body executes first unconditionally.
2. The loop test condition is evaluated at the bottom.

3. If the condition evaluates to true, a conditional branch jumps backward to the beginning of the body; otherwise, execution falls through.

Code Example 7.11

```
// C code:
long factorial_do_while(long n) {
    long result = 1;
    do {
        result *= n;
        n--;
    } while (n > 1);
    return result;
}

// x86-64 Assembly:
// Argument: n in %rdi
factorial_do_while:
    movq    $1, %rax            // result = 1
.L_body:
    imulq   %rdi, %rax          // result *= n
    subq    $1, %rdi            // n--
    cmpq    $1, %rdi            // Compare n : 1
    jg      .L_body             // If n > 1, jump back to .L_body
    ret                         // Return result
```

7.2.2 While Loops: Jump-to-Middle vs. Guarded-Do

Definition 7.12. While Loop Semantics: Unlike `do-while`, a standard `while (test) body` tests the condition *before* executing the loop body. If the initial test is false, the loop body must never execute. Compilers employ two distinct strategies to compile `while` loops:

Theorem 7.13. Strategy 1: Jump-to-Middle (JTM): Commonly generated by compilers at lower optimization levels (`-O0`):

- An initial unconditional jump (`jmp .L_test`) skips the loop body to the test located at the bottom.
- The test is evaluated, and if true, a conditional branch jumps back up to the body.
- **Characteristic:** Compact code footprint, but requires an extra unconditional branch upon initial entry.

Code Example 7.14

```
// Jump-to-Middle Pattern (GCC -O0 style):
    jmp     .L_test             // Jump to initial test
.L_body:
    // ... Loop Body Statements ...
```

```
.L_test:
    // ... Evaluate Condition ...
    j_cond  .L_body             // If condition holds, jump back to
        body
```

Theorem 7.15. Strategy 2: Guarded-Do (GD): Standard strategy generated by optimizing compilers (`-O1`, `-O2`, `-O3`):

- An initial conditional branch (the *guard*) tests the condition. If false, it jumps straight to the exit, bypassing the loop entirely.
- The remaining structure is compiled as an optimal **do-while loop**: the body runs, followed by a bottom-tested conditional branch.
- **Characteristic:** Steady-state loop iterations contain only a single conditional jump and zero unconditional jumps, maximizing branch predictor efficiency and loop throughput.

Code Example 7.16

```
// Guarded-Do Pattern (GCC -O1/O2 style):
    // ... Evaluate Condition ...
    j_not_cond .L_done          // Guard: if initially false, skip
        loop
.L_body:
    // ... Loop Body Statements ...
    // ... Evaluate Condition ...
    j_cond     .L_body          // If true, repeat loop
.L_done:
```

7.2.3 For Loops

Definition 7.17. For Loop Translation: The canonical C `for` loop:

```
for (init; test; update) {
    body;
}
```

is semantically equivalent to a `while` loop:

```
init;
while (test) {
    body;
    update;
}
```

Applying the guarded-do strategy yields:

```
init;
if (!test) goto done;
do {
    body;
```

```
update_label:
    update;
} while (test);
done:
```

Important 7.18. Semantics of `continue` in `for` Loops: In a `while` loop, executing `continue` jumps directly to the condition test. However, in a `for` loop, executing `continue` jumps directly to the **`update`** expression before testing the condition again.

7.3 Switch Statements and Multi-Way Branching

Definition 7.19. The C `switch` Statement: A `switch` provides multi-way branching based on the value of an integer index expression. Depending on the span and density of case constants, compilers select between two primary implementations:

1. **Jump Tables ($O(1)$ time):** Used when case values are dense (closely packed).
2. **Binary Decision Trees ($O(\log N)$ time):** Used when case values are sparse (widely separated).

7.3.1 Jump Tables for Dense Cases

Definition 7.20. Jump Table Architecture: A jump table is an array of code pointers (target memory addresses) stored in the read-only data section (`.rodata`) of memory:

- **Zero-Basing / Index Bias Subtraction:** If case constants start at min, the index is normalized by computing:

$$index = value - min$$

- **Single Unsigned Bounds Check:** To determine whether $index$ falls within $[0, span]$, the compiler performs an unsigned comparison against $span$:

`cmpq` $\$span$, $index$; `ja` .L_default

> **Remark 7.21. Unsigned Arithmetic Trick:** Because negative numbers are represented in two's complement, treating a negative $index$ as an unsigned integer turns it into an immense positive number ($> 2^{63}$). Consequently, a single unsigned check (`ja`) simultaneously catches values below min and values above max!

- **Indirect Jump:** Control is dispatched in constant $O(1)$ time using an indirect

jump with scaled index addressing:

$$\texttt{jmp} \quad *.\text{L_table}(,\%reg,8)$$

The processor reads the target 64-bit address from the jump table array at offset $8 \times index$ and loads it directly into `%rip`.

Code Example 7.22

```
// C code:
void switch_dense(long n, long *val) {
    switch (n) {
        case 100: *val += 10; break;
        case 102: *val += 20; // Fallthrough
        case 103: *val += 30; break;
        case 104: *val *= 2;  break;
        default:  *val = 0;   break;
    }
}

// x86-64 Assembly:
// Arguments: n in %rdi, val in %rsi
switch_dense:
    subq    $100, %rdi          // Zero-base: index = n - 100
    cmpq    $4, %rdi            // Check if index > 4 (span = 104 -
        100 = 4)
    ja      .L_default          // Unsigned > 4 covers n < 100 and
       n > 104
    jmp     *.L_jt(,%rdi,8)     // Indirect jump through table: %
       rip = .L_jt[index]

.section .rodata
    .align 8
.L_jt:
    .quad   .L_case100          // index 0 (n = 100)
    .quad   .L_default          // index 1 (n = 101, missing ->
       default)
    .quad   .L_case102          // index 2 (n = 102)
    .quad   .L_case103          // index 3 (n = 103)
    .quad   .L_case104          // index 4 (n = 104)

.text
.L_case100:
    addq    $10, (%rsi)
    ret
.L_case102:
    addq    $20, (%rsi)         // No break: falls through directly
        to .L_case103!
.L_case103:
    addq    $30, (%rsi)
    ret
.L_case104:
    shlq    $1, (%rsi)          // *val *= 2
    ret
.L_default:
```

```
42      movq    $0, (%rsi)
43      ret
```

Remark 7.23. Fallthrough and Missing Cases in Jump Tables:

- **Missing Cases:** Missing numbers within the case range (such as $n = 101$ above) simply point to the default label in the jump table array.
- **Fallthrough:** When a `break` is omitted, the code for one case immediately precedes the next case in the instruction stream, and execution simply falls through without a jump.

7.3.2 Sparse Switch Statements: Binary Decision Trees

Definition 7.24. Sparse Switch Compilation: When case constants are sparse and far apart (e.g., `case 5:`, `case 5000:`, `case 5000000:`), allocating a jump table would waste megabytes of memory on unused default entries. Instead, the compiler organizes the case labels into a balanced **Binary Decision Tree**:

- The compiler compares the input index against the median case value.
- It uses conditional jumps (`jl`, `jg`, `je`) to recursively partition the search space.
- **Time Complexity:** $O(\log N)$, where N is the number of case labels.
- For very small numbers of cases ($N \leq 3$), compilers simply emit a linear sequence of `cmp` and `je` comparisons ($O(N)$).

7.4 Procedures and the System V AMD64 ABI

Definition 7.25. Procedure Abstraction Mechanics: A procedure call requires orchestrating several low-level mechanisms between the **Caller** (invoking function) and the **Callee** (invoked function):

1. **Control Transfer:** Passing execution to the entry of the callee and cleanly resuming the caller upon completion.
2. **Data Transfer:** Passing parameters into the callee and returning a result value back to the caller.
3. **Memory Allocation:** Allocating private local storage for the callee upon entry and deallocating it upon exit.
4. **State Preservation:** Ensuring register contents relied upon by the caller are not corrupted by the callee.

All interactions conform to the **System V AMD64 ABI** on Linux and Unix systems.

7.4.1 The Runtime Stack and Frame Architecture

Definition 7.26. Stack Discipline: The runtime stack is allocated in high virtual memory and **grows downward** toward lower memory addresses:

- The stack pointer register `%rsp` holds the memory address of the current **top of the stack** (the lowest allocated valid byte).
- Pushing an 8-byte value: `pushq %reg` decrements `%rsp` by 8 and writes to `(%rsp)`.
- Popping an 8-byte value: `popq %reg` reads from `(%rsp)` and increments `%rsp` by 8.

Memory Region	**Contents and Purpose**
Caller Frame	High virtual memory addresses Incoming 8th argument (if present, pushed by caller) Incoming 7th argument (pushed by caller)
Frame Boundary	**Return Address** (pushed by `call` instruction)
Callee Frame	Saved caller base pointer `%rbp` (optional) Saved callee-saved registers (`%rbx`, `%r12`–`%r15`) Local stack variables, arrays, and structures Outgoing arguments (7+) passed to downstream functions
Top of Stack (`%rsp`)	Current lowest allocated stack address

Table 1: System V AMD64 Stack Frame Anatomy.

7.4.2 Control Transfer: `call` and `ret`

Definition 7.27. Control Transfer Primitives: Control transfer between caller and callee is managed via the hardware stack:

- **`call Target`:**
  1. Pushes the 8-byte return address (the address of the instruction immediately following `call`) onto the stack:
  $$\texttt{\%rsp} \leftarrow \texttt{\%rsp} - 8, \quad \text{Mem}[\texttt{\%rsp}] \leftarrow \texttt{\%rip}_{\text{next}}$$
  2. Jumps to the callee: `%rip` $\leftarrow$ Target.
- **`ret`:**
  1. Pops the return address off the top of the stack into `%rip`:
  $$\texttt{\%rip} \leftarrow \text{Mem}[\texttt{\%rsp}], \quad \texttt{\%rsp} \leftarrow \texttt{\%rsp} + 8$$
  2. Resumes execution in the caller at the instruction following the original `call`.

Important 7.28. The 16-Byte Stack Alignment Rule: The System V AMD64 ABI mandates that immediately before any `call` instruction is executed, the stack pointer `%rsp` must be a multiple of 16 (**16-byte aligned**).

- When the `call` executes, it pushes the 8-byte return address onto the stack.
- Therefore, upon function entry (at the first instruction of the callee), `%rsp` satisfies:

$$\texttt{\%rsp} \pmod{16} = 8$$

- If the callee makes further function calls, it must allocate sufficient stack space (e.g., `subq $8, %rsp` or an odd multiple of 8) so that `%rsp` is once again an exact multiple of 16 before issuing its own `call`.
- Failure to maintain 16-byte alignment causes crashes in SIMD/SSE instructions (e.g., `movaps`) that require 16-byte aligned memory.

7.4.3 Data Transfer: Arguments and Return Values

Theorem 7.29. Parameter Passing under System V AMD64 ABI: Function parameters and return values are passed via registers whenever possible to eliminate slow memory accesses:

- **First 6 Integer/Pointer Arguments:** Passed in dedicated general-purpose registers:

1st Arg	2nd Arg	3rd Arg	4th Arg	5th Arg	6th Arg
%rdi	%rsi	%rdx	%rcx	%r8	%r9

- **Arguments 7 and Higher:** Pushed onto the caller's stack frame in **reverse order** (argument 7 is at `(%rsp)` immediately before the `call`).
- **Floating-Point Arguments:** Passed in vector registers `%xmm0`–`%xmm7`.
- **Return Values:** Up to 64-bit integer/pointer return values are returned in `%rax`. 128-bit values use both `%rax` (low 64 bits) and `%rdx` (high 64 bits).

7.4.4 Caller-Saved vs. Callee-Saved Registers

Definition 7.30. Register Preservation Conventions: To prevent subroutines from inadvertently overwriting each other's live variables, all 16 general-purpose registers are categorized into two strict contractual sets:

Code Example 7.31

```
1 // Function demonstrating preservation of callee-saved registers:
2 compute:
3     pushq   %rbp                // Save caller's %rbp (callee-saved
          )
4     pushq   %rbx                // Save caller's %rbx (callee-saved
          )
5     subq    $8, %rsp            // Align stack to 16 bytes (pushed
          2*8=16 bytes, +8 ret = 24 -> need 8 more)
```

Category	Registers	Preservation Responsibility
Caller-Saved	`%rax`, `%rcx`, `%rdx`,	The **callee may freely overwrite** these registers
(Call-clobbered)	`%rsi`, `%rdi`, `%r8`,	(scratch). If the **caller** needs their values preserved
	`%r9`, `%r10`, `%r11`	across a function call, the caller must save them on its stack before the `call`.
Callee-Saved	`%rbx`, `%rbp`, `%rsp`,	The **callee is strictly obligated** to preserve these
(Call-preserved)	`%r12`, `%r13`, `%r14`,	registers. If the callee modifies any of them, it **must**
	`%r15`	push their original values upon entry and restore them before returning.

Table 2: Classification of x86-64 Registers by Preservation Responsibility.

```
6
7       movq    %rdi, %rbx          // Safely store argument in callee-
            saved %rbx
8       call    helper_func         // helper_func cannot clobber %rbx!
9       addq    %rbx, %rax          // Add preserved value to helper
            return value
10
11      addq    $8, %rsp            // Deallocate local alignment
            padding
12      popq    %rbx                // Restore caller's %rbx
13      popq    %rbp                // Restore caller's %rbp
14      ret                         // Return to caller
```

7.4.5 Local Storage on the Stack and the Red Zone

Definition 7.32. When Local Variables Must Reside on the Stack: Although compilers place local variables in registers whenever possible, memory on the stack is required when:

1. **Insufficient Registers:** Too many local variables are active simultaneously (*register spilling*).
2. **Address-of Operator (`&`):** If the code evaluates `&x`, the variable x must have a concrete physical memory address (registers do not have memory addresses).
3. **Arrays and Structures:** Contiguous buffers or aggregate data types accessed via index arithmetic.

Definition 7.33. The Red Zone Optimization: The x86-64 System V ABI specifies a 128-byte region beyond the current stack pointer (`-128(%rsp)` to `-1(%rsp)`) known as the **Red Zone**:

- This space is guaranteed **not** to be clobbered by operating system signals or interrupt handlers.
- **Leaf Functions** (functions that do not invoke any other functions) can use this 128-byte memory space for local storage without adjusting the stack pointer (`subq / addq %rsp`), eliminating stack allocation overhead.

7.4.6 Variadic Functions (`stdarg.h`)

Definition 7.34. Variadic Functions: A **variadic function** is a function that accepts a variable number of arguments of differing types (e.g., `int printf(const char *format, ...)`).

Theorem 7.35. Variadic Calling Convention in x86-64 ABI: Because the callee does not know ahead of time how many floating-point arguments were passed, the System V ABI enforces a special register convention:

- **Role of `%rax`:** Before calling a variadic function, the caller **must set `%rax` (or `%al`) to the number of vector registers (`%xmm0`–`%xmm7`) used** to pass floating-point arguments.
- For standard calls with no floating-point arguments, the caller emits:

  `movl $0, %eax` or `xorl %eax, %eax`

- **Register Save Area (RSA):** Upon entry, a variadic callee uses the value in `%al` to selectively dump the incoming argument registers into a contiguous block on its stack frame.
- **The `va_list` Cursor:** The `va_list` type is an internal structure that records offsets into the Register Save Area and the stack argument overflow area, enabling the `va_arg()` macro to retrieve each consecutive argument.

Code Example 7.36

```
// Calling printf in x86-64 assembly:
.section .rodata
.L_fmt:
    .string "Sum: %ld, Float: %f\n"
.align 8
.L_val:
    .double 3.1415926535

.text
.globl main
main:
    subq    $8, %rsp            // Align stack to 16 bytes
    leaq    .L_fmt(%rip), %rdi  // 1st argument (format string) in
        %rdi
    movq    $42, %rsi           // 2nd argument (integer %ld) in %
        rsi
```

```
15      movsd   .L_val(%rip), %xmm0 // 3rd argument (double %f) in %
            xmm0
16      movl    $1, %eax            // Exactly 1 vector register (%xmm0
            ) used!
17      call    printf              // Invoke variadic printf
18      xorl    %eax, %eax          // Return 0
19      addq    $8, %rsp            // Restore stack pointer
20      ret
```